Laborator 1
Introducere în PL/SQL
Variabile

I. Obiective

1. Introducere în PL/SQL

1.1. Caracteristici generale
1.2. Blocuri PL/SQL
1.3. Operatori în PL/SQ
1.4. Funcţii SQL suportate în PL/SQL
1.5. Convenţii în blocuri PL/SQL
1.6. Comenzi SQL suportate în PL/SQL

2. Variabile

2.1 Declarare şi iniţializare
2.2 Tipuri de variabile
2.3 Atributul %TYPE
2.4 Variabile de mediu sau variabile de legătură ale aplicaţiilor gazdă (BIND VARIABLES)
2.5 Variabile de substituţie

II. Consideraţii teoretice

1. Introducere în PL/SQL

1.1. Caracteristici generale:

- Construcţiile PL/SQL conţin structuri de control procedurale şi comenzi descriptive SQL
- PL/SQL este un limbaj procedural structurat pe bloc, programele putând fi împărţite în blocuri logice
- Blocurile PL/SQL sunt procesate de motorul PL/SQL care poate fi rezident pe ORACLE SERVER sau pe un instrument de dezvoltare (ex.: Oracle Forms, Reports, JDeveloper etc.)
- Multe instrumente ORACLE au propriul motor PL/SQL (ex.: Oracle Forms, Reports, JDeveloper etc.)
- Tipurile de date din SQL pot fi folosite în PL/SQL
- Programarea în PL/SQL este modularizată – se utilizează blocurile care grupează instrucţiunile

1.2. Blocuri PL/SQL

Orice unitate PL/SQL conţine unul sau mai multe blocuri, complet separate sau imbricate.

Componentele unui bloc PL/SQL:

Un bloc PL/SQL este compus din până la 3 secţiuni: declarativă (opţională), executabilă (obligatorie) şi de tratare a excepţiilor (opţională).

DECLARE *(Opţional)*
variabile, cursori, excepţii
BEGIN *(Obligatoriu)*
comenzi SQL (asigură accesul la baza de date)
structuri de programare procedurală PL/SQL
EXCEPTION *(Opţional)*
acţiuni ce se execută când apare o eroare
END; *(Obligatoriu)*

Observaţii:

- comenzile SQL asigură accesul la baza de date;
- operaţiile efectuate cu variabilele PL/SQL în cadrul instrucţiunilor procedurale nu presupun accesarea bazei de date;
- se foloseşte (**;**) după fiecare instrucţiune SQL sau instrucţiune de control PL/SQL;
- blocul PL/SQL se termină cu (**;**)
- se foloseşte (/) pentru a lansa un bloc anonim în bufferul SQL;
- o eroare în PL/SQL este tratată ca o excepţie.

Tipuri de blocuri PL/SQL:

Blocuri anonime:

1. Funcţii stocate şi funcţii de aplicaţii
2. Proceduri stocate şi proceduri de aplicaţii
3. Pachete
4. Declanşatoare (triggeri) pe baza de date / de aplicaţii

Blocurile anonime:

- sunt nedenumite
- nu sunt stocate în baza de date
- se declară în linie, în locul în care se doreşte execuţia lor
- se execută în momentul rulării

Exemplu:

```
DECLARE
        v_variabila varchar2(5);
BEGIN
        SELECT coloana
        INTO v_variabila
        FROM tabela;
        EXCEPTION
                WHEN excepţie THEN acţiune
END;
/
```

Blocuri anonime imbricate

- se pot imbrica mai multe blocuri;
- acestea se pot eticheta cu <<**eticheta_bloc**>> , iar variabilele din cadrul blocurilor se pot utiliza astfel:

eticheta_bloc.variabila.

```
BEGIN
…….

<< eticheta_bloc >>
DECLARE
        ……..
BEGIN
        ……..

END eticheta_bloc;
END;

/
```

Proceduri, funcţii:

- blocuri PL/SQL cu un nume;
- se pot stoca la nivel de ORACLE SERVER (proceduri/funcţii stocate) sau la nivel de aplicaţie (DEVELOPER – Forms și Reports).

Exemple:

```
CREATE [OR REPLACE] PROCEDURE nume_procedura IS
…….
BEGIN
…….
[EXCEPTION]
…….
END;
/

CREATE [OR REPLACE] FUNCTION nume_functie
RETURN tip_data
IS
BEGIN
……
RETURN valoare
[EXCEPTION]
……
END;
/
```

Pachete de programe - grupează proceduri, funcţii.

Declanşatori pe baza de date - blocuri PL/SQL asociate tabelelor (de bază sau virtuale) şi lansate automat în execuţie când are loc o comandă de manipulare.

Declanşatori de aplicaţie - blocuri PL/SQL asociate unor evenimente din cadrul aplicaţiei (de exemplu: deplasarea mouse-ului, apăsarea unui buton) şi lansate în execuţie automat.

1.3 Operatori în PL/SQL

Operator	Caracteristici
+, -, *, /, ** (op. exponenţial)	Operatori aritmetici
AND, OR, NOT	Operatori logici
<, >, =, >=, <=, <>, !=	Operatori de comparaţie
BETWEEN ... AND ...	Operator de verificare a apartenenţei la un interval
IN(listă)	Operator de verificare a apartenenţei la o listă de valori
LIKE	Operator de comparare cu un şablon **%** - oricâte caractere; **_** - un caracter;
IS NULL	Operator care verifică dacă o variabilă are valoarea NULL
||	Operator de concatenare
@	Operator de conectare la distanţă
& **sau &&**	Operatori pentru adresarea variabilelor de substituţie
:=	Operator de atribuire

1.4 Funcţii SQL suportate în PL/SQL

- În cadrul instrucţiunilor descriptive sunt suportate toate tipurile de funcţii SQL (inclusiv funcţiile de grup în cadrul instrucţiunii SELECT);
- Instrucţiunile PL/SQL:
  - ✓ Suportă funcţii la nivel de înregistrare (single-row): numerice, caracter, data, de conversie etc.;
  - ✓ **NU** suportă funcţii de grup (SUM, MIN, MAX, AVG, COUNT, STDDEV) sau funcţia DECODE. De exemplu nu se pot utiliza construcţii de forma: *IF DECODE(...) THEN ...* sau *IF AVG(...) THEN ...*

1.5 Conversii în blocurile PL/SQL

- PL/SQL converteşte tipurile de date dinamic (de exemplu: o valoare numerică la o variabilă char);
- conversii implicite: caracter <-> numeric si caracter <->data;
- conversii explicite: se utilizează funcţiile TO_DATE, TO_NUMBER, TO_CHAR.

1.6 Comenzi SQL suportate în PL/SQL

- PL/SQL permite folosirea comenzilor de manipulare a datelor (LMD):
  - ✓ SELECT
  - ✓ INSERT
  - ✓ UPDATE
  - ✓ DELETE
- PL/SQL permite folosirea comenzilor de control al tranzacţiilor:
  - ✓ COMMIT
  - ✓ ROLLBACK

- ✓ SAVEPOINT

Notă: Un bloc PL/SQL nu este o tranzacţie. Comenzile Commit/ Rollback/ Savepoint sunt independente de bloc, dar pot să apară în cadrul acestuia.

- PL/SQL **NU** suportă comenzile de definire a datelor (LDD)
  - ✓ CREATE
  - ✓ ALTER
  - ✓ DROP
  - ✓ RENAME
  - ✓ TRUNCATE
- PL/SQL **NU** suportă comenzile din cadrul limbajului pentru controlul datelor (Data Control Language - DCL)
  - ✓ GRANT
  - ✓ REVOKE

2. Variabile

2.1 Declarare şi iniţializare

- declararea variabilelor se realizează în zona declarativă (delimitată prin DECLARE) a blocului (sau sub-blocului);
- iniţializarea se poate face la declarare sau în zona de execuţie (între BEGIN şi END);
- variabilele vor fi vizibile în restul blocului, respectiv şi în blocurile incluse în el, mai puţin în sub-blocurile în care numele lor este redefinit (ca în majoritatea limbajelor de programare structurate, semnificaţia unui nume definit de utilizator într-un bloc/sub-bloc este dată de cea mai apropiată declaraţie anterioară locului folosirii);
- toate variabilele PL/SQL au un tip de dată, restricţii şi un şir valid de valori;
- declararea şi iniţializarea se realizează astfel:

nume_variabila [CONSTANT] TIP_DATA [NOT NULL] [:= |DEFAULT expresie]

- constantele trebuie obligatoriu iniţializate, iar ulterior nu îşi vor putea schimba valoarea;
- variabilele NOT NULL trebuie obligatoriu iniţializate, iar ulterior nu vor putea primi valoarea NULL;
- se foloseşte următoarea convenţie de notare:

  c_nume Constanta
  v_nume Variabila
  g_nume VarGlobala (variabilă globală definită în zona de specificaţii a pachetului de programe şi valabilă pentru toate subprogramele pachetului).

2.2 Tipuri de variabile

Variabile PL/SQL

- **Scalare**
- **Compozite**
- **Referinţă**
- **LOB (Large Objects): NCLOB, CLOB, BLOB, BFILE**
- **Obiect**

Variabile non-PL/SQL: variabile de mediu (BIND VARIABLES)

Variabile Scalare:

Tipurile scalare conţin valori simple (o variabilă scalară poate conţine la un moment dat o singură valoare simplă) şi corespund în principal tipurilor pe care le pot avea coloanele tabelelor.

- char (lung_max) - lungime fixă de max 32.767 bytes
- varchar2 (lung_max) – lungime variabilă de max 32.767 bytes
- long [şir de caractere de lungime variabilă 2GB]
- number (precizie,scală)
- boolean (true, false, null)
- date
- binary_integer şi pls_integer (numere întregi între -2147483647 şi 2147483647)
- binary_float şi binary_double (pentru numere reale în varianta Oracle 10g)
- timestamp (pentru fracţiuni de secundă)

Exemple:

```
v_functie varchar2(9);
v_numar binary_integer:=0;
v_totalsal number(9,2):=0;
v_datainceput date:=sysdate+7;
c_taxa constant number(3,2):=8.25;
```

Afişarea variabilelor PL/SQL se realizează prin intermediul funcţiei PUT_LINE din pachetului DBMS_OUTPUT. Se poate utiliza operatorul de concatenare (||) pentru a afişa mai multe mesaje sau variabile pe aceeaşi linie.

```
DBMS_OUTPUT.PUT_LINE ('VALOAREA VARIABILEI ESTE:' ||variabila);
```

Popularea variabilelor cu valori din tabelele bazei de date:

- Se utilizează comanda SELECT cu clauza INTO pentru popularea variabilelor PL/SQL cu valori ale atributelor din tabele;

- cererile SELECT din cadrul blocurilor PL/SQL trebuie să furnizeze o singură linie rezultat (în caz contrar se semnalează eroare).

Exemplu:

```
--se afișează numele angajatului cu codul 100
SET SERVEROUTPUT ON
DECLARE
v_nume VARCHAR2(20);
BEGIN
      SELECT nume
      INTO v_nume
      FROM angajati
      WHERE id_angajat = 100;
      DBMS_OUTPUT.PUT_LINE('NUMELE ANGAJATULUI ESTE:' ||v_nume);
  END;
/v_valid boolean not null:=true;
```

2.3. Atributul %TYPE

Atribuie unei variabile tipul altei variabile sau tipul de date specific unei coloane din tabelă.

Declararea unei variabile cu %TYPE:

```
variabila     tabelă.nume_coloană%TYPE;
                    sau
variabila1   tip_dată;
variabila2   variabila1%TYPE;
```

Exemplu:

```
--afișează numele și prenumele angajatului cu codul 100.
DECLARE
        v_nume angajati.nume%TYPE;
        v_prenume angajati.prenume%TYPE;
BEGIN
        SELECT nume, prenume
        INTO v_nume, v_prenume
        FROM angajati
        WHERE id_angajat = 100;
        DBMS_OUTPUT.PUT_LINE('NUMELE ANGAJATULUI ESTE:' ||
        v_nume||' '||v_prenume);
END;
/v_valid boolean not null:=true;
```

Observaţie: Restricţia NOT NULL a unei coloane nu se aplică şi variabilei declarate prin folosirea atributului %TYPE.

2.4. Variabile de mediu sau variabile de legătură ale aplicaţiilor gazdă (BIND VARIABLES)

- sunt variabile de legătură cu aplicaţia în care rulează motorul PL/SQL;
- trebuie declarate în aplicaţie (în mediul gazdă) şi pot fi accesate şi modificate în cadrul blocurilor PL/SQL;
- după terminarea execuţiei blocului PL/SQL, variabila rămâne în mediul gazdă cu valoarea primită în urma rulării blocului (şi poate fi pasată altui bloc, realizând astfel transmiterea de valori între blocurile PL/SQL);
- nu pot fi utilizate în cadrul procedurilor, funcţiilor sau pachetelor;
- se declară în afara blocului PL/SQL cu ajutorul cuvântului cheie VARIABLE:

  VARIABLE g_numevariabilă TIP
- pentru declararea unei variabile host de tip NUMBER nu se specifică precizia şi scala;
- pentru utilizarea lor în cadrul unui bloc PL/SQL sau într-o frază SQL din afara blocului se prefixează cu ":"

  :host_variabila:=v_variabila;
- se afişează în afara blocului cu ajutorul comenzii PRINT (la afişare variabila nu se va prefixa cu ":")

  PRINT g_numevariabilă

Exemple:

```
--se afişează mesajul: Bloc PL/SQL
VARIABLE g_mesaj varchar2(30)
BEGIN
        :g_mesaj:='Bloc PL/SQL';
END;
/
PRINT g_mesaj
```

```
SET SERVEROUTPUT ON
        VARIABLE g_comenzi varchar2(30)
BEGIN
        select count(*) into :g_comenzi
        from comenzi
        where modalitate = 'online';
END;
/
```

PRINT g_comenzi

*Observaţi*e: Se poate auto-afişa variabila prin setarea AUTOPRINT ON

Exemplu:

```
--se selecteaza produsele si pretul acestora pentru acele produse care au pretul <
pretul mediu al produsului cu codul 3133 fara a utiliza un select imbricat

SET SERVEROUTPUT ON
SET AUTOPRINT ON
VARIABLE g_pret number

BEGIN
        select avg(pret) into :g_pret
        from rand_comenzi
        where id_produs = 3133;
END;
/
select * from rand_comenzi where pret< :g_pret;
```

2.5. Variabile de substituţie

- de regulă, variabilele de substituţie sunt folosite pentru a transmite valori dinspre mediul SQL*Plus spre comenzile SQL sau blocurile PL/SQL, în timp ce variabilele de legătură (bind variables) sunt folosite pentru a transmite valori în sens invers sau pentru a transfera valori între blocuri PL/SQL lansate succesiv (primul bloc setează variabila, următorul o consultă)
- se pot invoca din comenzile SQL sau din blocurile PL/SQL prin "&nume_variabila" sau "&&nume_variabila"
- sunt locale sesiunii SQL în care au fost declarate
- în mediul SQL variabilele de substituţie pot fi uşor citite prin introducerea de valori de la tastatură (utilizând ACCEPT), se pot defini (cu DEFINE) sau afişa pe ecran (cu PROMPT)

Exemple:

```
--se afiseaza numarul de comenzi ale angajatului al carui cod este introdus de
utilizator prin intermediul variabilei de substitutie &id_angajat DECLARE

        v_nr_comenzi number(2);
BEGIN
        select count(nr_comanda)
        into v_nr_comenzi from comenzi
        where id_angajat=&id_angajat;
        dbms_output.put_line('Angajatul are: '|| v_nr_comenzi||' comenzi');
END;
/
```

Observaţie: Într-un bloc PL/SQL se pot utiliza toate tipurile de variabile, respectând însă caracteristicile şi regulile de utilizare ale acestora.

În exemplul următor se utilizează atât variabila de substituţie s_nume definită şi iniţializată prin comanda DEFINE, cât şi variabila de legătură g_salariul, dar şi variabila locală v_prenume de acelaşi tip cu coloana nume din tabela Angajati. Variabila de substituţie definită cu DEFINE va fi implicit de tipul CHAR:

```
-- se afiseaza salariul si prenumele angajatului cu numele Abel
SET SERVEROUTPUT ON VARIABLE g_salariul number
DEFINE s_nume=Abel
DECLARE
    v_prenume angajati.nume%type;
BEGIN
    select prenume,salariul into v_prenume, :g_salariul from angajati where
    nume='&s_nume';
    DBMS_OUTPUT.PUT_LINE ('Prenumele angajatului este: '||v_prenume);
END;
/
print g_salariul
```

III. Desfășurarea lucrării

- Familiarizare cu mediul limbajului PL/SQL.

Probleme propuse spre rezolvare

1. Specificaţi ce se va afişa la rularea următorului bloc PL/SQL:

```
DECLARE
        v_var1 NUMBER :=100;
```

```
        v_var2 NUMBER;
        v_var3 NUMBER := v_var2;
        v_var4 VARCHAR(20) := 'variabila PL/SQL';
        v_var5 NUMBER NOT NULL := v_var1;
        c_const1 CONSTANT DATE := TO_DATE('12/02/2007','dd/mm/yyyy');
        c_const2 CONSTANT NUMBER NOT NULL := 2;
        c_const3 CONSTANT NUMBER := NULL;
        v_var6 NUMBER DEFAULT NULL;
BEGIN
        DBMS_OUTPUT.PUT_LINE('variabila 1 = '||v_var1);
        DBMS_OUTPUT.PUT_LINE('variabila 2 = '||v_var2);
        DBMS_OUTPUT.PUT_LINE('variabila 3 = '||v_var3);
        DBMS_OUTPUT.PUT_LINE('variabila 4 = '||v_var4);
        DBMS_OUTPUT.PUT_LINE('variabila 5 = '||v_var5);
        DBMS_OUTPUT.PUT_LINE('constanta 1 = '||c_const1);
        DBMS_OUTPUT.PUT_LINE('constanta 2 = '||c_const2);
        DBMS_OUTPUT.PUT_LINE('constanta 3 = '||c_const3);
        DBMS_OUTPUT.PUT_LINE('variabila 6 = '||v_var6);
END;
/
```

2. Specificaţi ce se va afişa la rularea următorului bloc PL/SQL (care conţine blocuri imbricate, ilustrând domeniul de vizibilitate al unor variabile care au acelaşi nume):

```
DECLARE
        var NUMBER;
BEGIN
        var := 1;
        DBMS_OUTPUT.PUT_LINE(var);

        <<bloc1>>
        DECLARE
                var NUMBER;
        BEGIN
                var :=2;
                DBMS_OUTPUT.PUT_LINE(var);
        END bloc1;

        DBMS_OUTPUT.PUT_LINE(var);

        <<bloc2>>
        DECLARE
                var NUMBER;
        BEGIN
                var :=3;
                DBMS_OUTPUT.PUT_LINE(var);

        <<bloc3>>
        DECLARE
                var NUMBER;
                BEGIN
                        var :=4;
                        DBMS_OUTPUT.PUT_LINE(var);
                        DBMS_OUTPUT.PUT_LINE(bloc2.var);
        END bloc3;
```

```
        DBMS_OUTPUT.PUT_LINE(var);
        END bloc2;
        DBMS_OUTPUT.PUT_LINE(var);
END;
/
```

3. Specificaţi ce se va afişa la rularea următorului bloc PL/SQL

```
DECLARE
        stoc NUMBER(3):=600;
        mesaj VARCHAR2(50):='Produsul 101';
BEGIN
        DECLARE
                stoc NUMBER(3):=10;
                mesaj VARCHAR2(50):='Produsul 102';
                um VARCHAR2(10):= ' bucati ';
        BEGIN
                stoc:= stoc+1;
                mesaj:='Stocul pentru '||mesaj||' este de: '||stoc||um;
                DBMS_OUTPUT.PUT_LINE(mesaj);
        END;
        stoc:= stoc+100;
        mesaj:='Stocul pentru '||mesaj||' este de: '||stoc||um;
        DBMS_OUTPUT.PUT_LINE(mesaj);
END;
/
```

4. Să se calculeze suma a două numere, iar rezultatul să se dividă cu 3. Numerele se vor introduce de la tastatură.

```
VARIABLE g_rezultat number
        ACCEPT p_num1 PROMPT 'Introduceţi primul număr:'
        ACCEPT p_num2 PROMPT 'Introduceţi al doilea număr:'
DECLARE
        v_num1 number(9,2):=&p_num1;
        v_num2 number(9,2):=&p_num2;
BEGIN
        :g_rezultat:=(v_num1+v_num2)/3;
        END;
/
PRINT g_rezultat
```

5. Să se afişeze salariul mărit cu un procent. Salariul şi procentul se dau de la tastatură

```
ACCEPT p_sal PROMPT 'Introduceţi salariul:' ACCEPT
p_procent PROMPT 'Introduceţi procentul:'
DECLARE
        v_sal number:=&p_sal;
        v_procent number:=&p_procent;
BEGIN
        dbms_output.put_line(to_char(nvl(v_sal,0)*(1+nvl(v_procent,0)/100)));
END;
/
```